JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Spliterator.OfDouble

All Superinterfaces:

Spliterator<Double>, Spliterator.OfPrimitive<Double, DoubleConsumer, Spliterator.OfDouble>

All Known Implementing Classes:

Spliterators.AbstractDoubleSpliterator

Enclosing interface:

Spliterator<T>

---

public static interface **Spliterator.OfDouble**
extends Spliterator.OfPrimitive<Double, DoubleConsumer, Spliterator.OfDouble>

A Spliterator specialized for double values.

Since:

1.8

Nested Class Summary

Nested classes/interfaces declared in interface Spliterator
Spliterator.OfDouble, Spliterator.OfInt, Spliterator.OfLong, Spliterator.OfPrimitive<T,T_CONS,T_SPLITR>

Field Summary

Fields declared in interface Spliterator
CONCURRENT, DISTINCT, IMMUTABLE, NONNULL, ORDERED, SIZED, SORTED, SUBSIZED

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
default void	forEachRemaining(Consumer<? super Double> action)	Performs the given action for each remaining element, sequentially in the current thread, until all

		elements have been processed or the action throws an exception.
default boolean	**tryAdvance**(Consumer<? super Double> action)	If a remaining element exists: performs the given action on it, returning `true`; else returns `false`.

Methods declared in interface Spliterator

characteristics, estimateSize, getComparator, getExactSizeIfKnown, hasCharacteristics, trySplit

Methods declared in interface Spliterator.OfPrimitive

forEachRemaining, tryAdvance

Method Details

tryAdvance

```
default boolean tryAdvance(Consumer<? super Double> action)
```

If a remaining element exists: performs the given action on it, returning `true`; else returns `false`. If this Spliterator is `Spliterator.ORDERED` the action is performed on the next element in encounter order. Exceptions thrown by the action are relayed to the caller.

Subsequent behavior of a spliterator is unspecified if the action throws an exception.

Specified by:

tryAdvance in interface Spliterator<Double>

Implementation Requirements:

If the action is an instance of `DoubleConsumer` then it is cast to `DoubleConsumer` and passed to `Spliterator.OfPrimitive.tryAdvance(java.util.function.DoubleConsumer)`;

otherwise the action is adapted to an instance of `DoubleConsumer`, by boxing the argument of `DoubleConsumer`, and then passed to `Spliterator.OfPrimitive.tryAdvance(java.util.function.DoubleConsumer)`.

Parameters:

`action` - The action whose operation is performed at-most once

Returns:

`false` if no remaining elements existed upon entry to this method, else `true`.

forEachRemaining

```
default void forEachRemaining(Consumer<? super Double> action)
```

Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action throws an exception. If this Spliterator is `Spliterator.ORDERED`, actions are performed in encounter order. Exceptions thrown by the action are relayed to the caller.

Subsequent behavior of a spliterator is unspecified if the action throws an exception.

Specified by:

`forEachRemaining` in interface `Spliterator<Double>`

Implementation Requirements:

If the action is an instance of `DoubleConsumer` then it is cast to `DoubleConsumer` and passed to `Spliterator.OfPrimitive.forEachRemaining(java.util.function.DoubleConsumer)`; otherwise the action is adapted to an instance of `DoubleConsumer`, by boxing the argument of `DoubleConsumer`, and then passed to `Spliterator.OfPrimitive.forEachRemaining(java.util.function.DoubleConsumer)`.

Parameters:

`action` - The action

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.